

《算法设计技巧与分析》

第7章 贪心算法

朱友文

zhuyw@nuaa.edu.cn

继续看背包问题

物品编号	价值	体积	价值/体积
1	28	4	7
2	20	3	6.67
3	11	2	5.5
4	1	1	1

背包体积为 5，求能够装入的物品的总价值的最大值

排序后放入的思路，实际上是，使得已占用的体积价值最大

贪心算法：从局部最优出发

7.1 引言

➤ 贪心策略:

➤ 每一步都建立在局部最优解的基础上

➤ 不一定可以获得全局最优解

➤ 有些问题可以获得全局最优解:

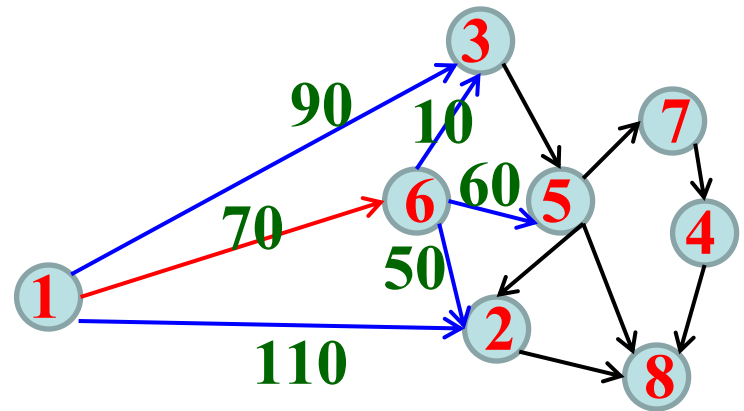
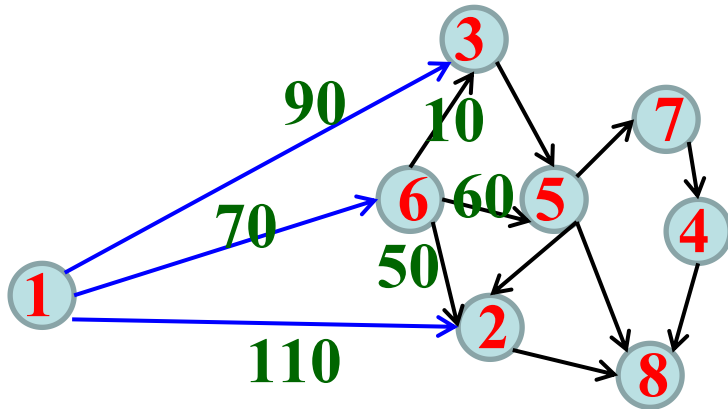
比如: 最短路径问题、最小生成树.....

7.2 最短路径

➤ 问题： $G=(V, E)$ 有向带权图（权值非负）

➤ Dijkstra算法

源节点1； $\lambda[n]$



算法框架

1. $X \leftarrow \{1\}; Y \leftarrow V - \{1\}$
2. 对每个 $v \in Y$, 如果存在从 1 到 v 的边, 则令 $\lambda[v]$ (v 的标记) 为边的长度; 否则令 $\lambda[v] = \infty$, 并设 $\lambda[1] = 0$
3. **while** $Y \neq \{\}$
4. 令 $y \in Y$, 使得 $\lambda[y]$ 为最小
5. 将 y 从 Y 移到 X
6. 更新那些在 Y 中与 y 相邻的顶点的标记
7. **end while**

➤ Dijkstra算法 正确性

归纳法

离得近的先加入

正确性分析

➤ 记 $\lambda[1...n]$ 为Dijkstra算法计算的结果

➤ 记 $\delta[1...n]$ 为真正的最短路径长度

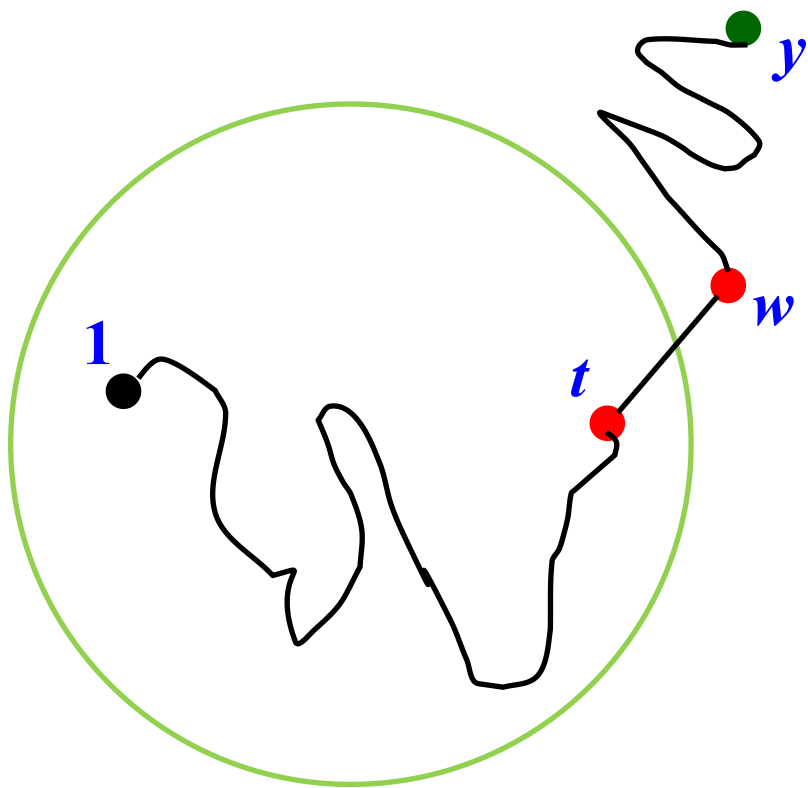
归纳法：

- ✓ 显然，第一个加入的 $\lambda[1] = \delta[1] = 0$ ；
- ✓ 第二个加入的也满足 $\lambda[v] = \delta[v]$ ；
- ✓ 加上前 k 个加入的都满足 $\lambda[v] = \delta[v]$ ，那么对于第 $k+1$ 个加入的点 y ，我们可以证明 $\lambda[y] = \delta[y]$ 也成立
- ✓ 反证法：如果 $\lambda[y] > \delta[y]$ ，那么 $\delta[y]$ 对应的1到 y 的路径中至少包括一个尚未加入 X 集合的点 w ，

正确性分析

归纳法:

- ✓ 反证法: 如果 $\lambda[y] > \delta[y]$, 那么 $\delta[y]$ 对应的1到y的路径中至少包括一个尚未加入X集合的点w, 假设 $\delta[y]$ 对应的路径是 $\langle 1 \dots t, w, \dots y \rangle$



那么

$$\begin{aligned}\lambda[y] &\leq \lambda[w] \\ &\leq \lambda[t] + \text{length}(t, w) \\ &= \delta[t] + \text{length}(t, w) \\ &= \delta[w] \\ &\leq \delta[y] \quad \text{得到矛盾}\end{aligned}$$

因此 $\lambda[y] \leq \delta[y]$

1. $X = \{1\}; Y \leftarrow V - \{1\}; \lambda[1] \leftarrow 0$

2. **for** $y \leftarrow 2$ **to** n

3. **if** y 相邻于 1 **then** $\lambda[y] \leftarrow \text{length}[1, y]$

4. **else** $\lambda[y] \leftarrow \infty$

5. **end if**

6. **end for**

7. **for** $j \leftarrow 1$ **to** $n - 1$

8. 令 $y \in Y$, 使得 $\lambda[y]$ 为最小

9. $X \leftarrow X \cup \{y\}$ {将顶点 y 加入 X }

10. $Y \leftarrow Y - \{y\}$ {将顶点 y 从 Y 中删除}

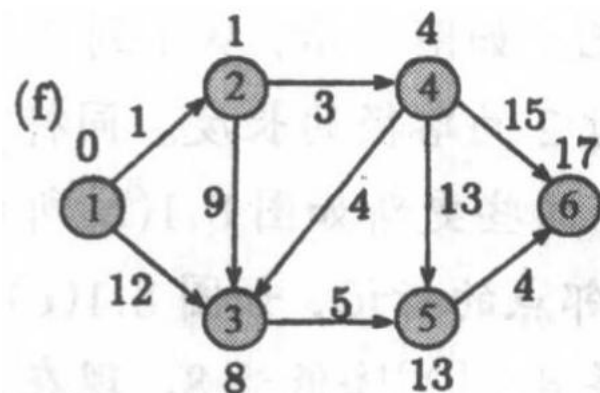
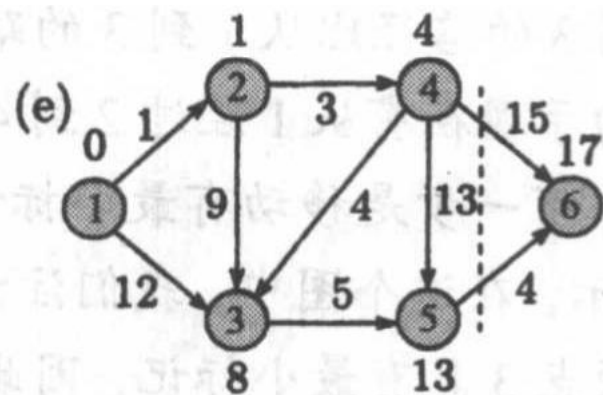
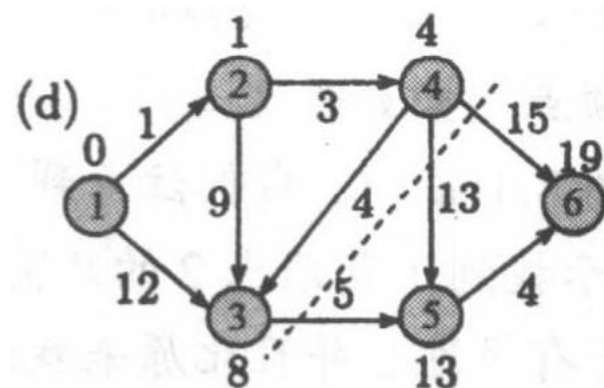
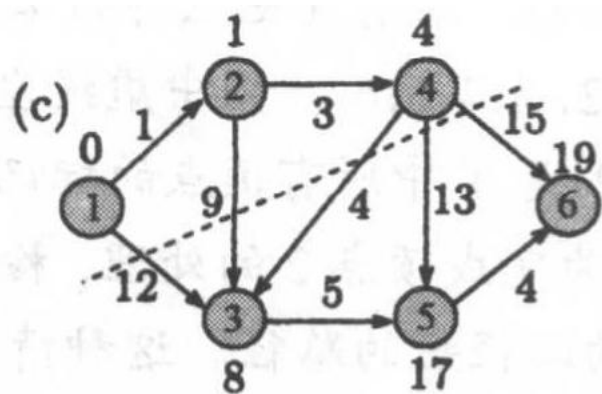
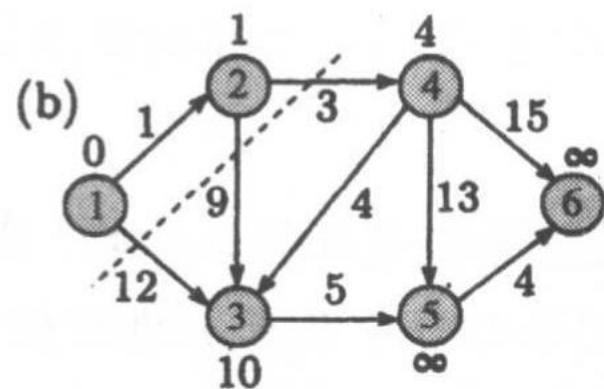
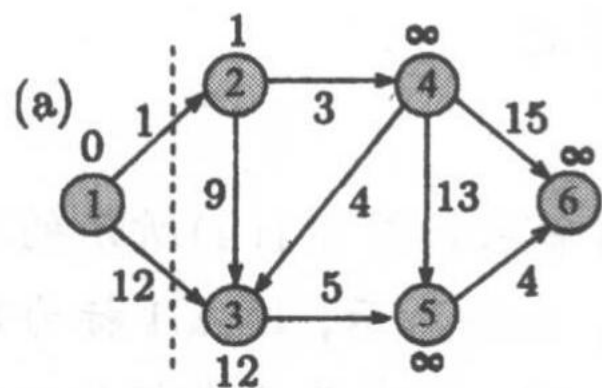
11. **for** 每条边 (y, w)

12. **if** $w \in Y$ **and** $\lambda[y] + \text{length}[y, w] < \lambda[w]$ **then**

13. $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$

14. **end for**

15. **end for**



7.2 最短路径

➤ 问题： $G=(V, E)$ 有向带权图

➤ Dijkstra 算法

时间复杂性： 分别计算每步的计算代价

1. $X = \{1\}; Y \leftarrow V - \{1\}; \lambda[1] \leftarrow 0$

2. **for** $y \leftarrow 2$ **to** n

3. **if** y 相邻于 1 **then** $\lambda[y] \leftarrow \text{length}[1, y]$

4. **else** $\lambda[y] \leftarrow \infty$

5. **end if**

6. **end for**

$O(n)$

7. **for** $j \leftarrow 1$ **to** $n - 1$

8. 令 $y \in Y$, 使得 $\lambda[y]$ 为最小

9. $X \leftarrow X \cup \{y\}$ {将顶点 y 加入 X }

10. $Y \leftarrow Y \cup \{y\}$ {将顶点 y 从 Y 中删除}

$O(n^2)$

11. **for** 每条边 (y, w)

12. **if** $w \in Y$ **and** $\lambda[y] + \text{length}[y, w] < \lambda[w]$ **then**

13. $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$

14. **end for**

$O(m)$

15. **end for**

1. $X = \{1\}; Y \leftarrow V - \{1\}; \lambda[1] \leftarrow 0$

2. **for** $y \leftarrow 2$ **to** n

3. **if** y 相邻于 1 **then** $\lambda[y] \leftarrow \text{length}[1, y]$

4. **else** $\lambda[y] \leftarrow \infty$

5. **end if**

6. **end for**

$O(n)$

7. **for** $j \leftarrow 1$ **to** $n - 1$

8. 令 $y \in Y$, 使得 $\lambda[y]$ 为最小

9. $X \leftarrow X \cup \{y\}$ {将顶点 y 加入 X }

10. $Y \leftarrow Y \cup \{y\}$ {将顶点 y 从 Y 中删除}

$O(n^2)$

11. **for** 每条边 (y, w)

12. **if** $w \in Y$ **and** $\lambda[y] + \text{length}[y, w] < \lambda[w]$ **then**

13. $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$

14. **end for**

$O(m)$

15. **end for**

3.3 不相交集数据结构

- 考虑一种场景：全集 A 包含 n 个元素 $\{1, 2, \dots, n\}$ ，这些节点初始时分成 n 个不相交的子集

$$A = \bigcup_{i=1}^k S_i \quad \text{其中 } S_i = \{i\}$$

- 需要频繁执行两种操作：
- 子集合并
 - 判断两个元素是否属于同一个子集

3.3 不相交集数据结构

➤ 不相交集数据结构的几种基本操作:

- **MAKESET()**
- **UNION(x, y)**
- **FIND(x)**

3.3 不相交集数据结构

MAKESET() //初始状态

0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10

3.3 不相交集数据结构

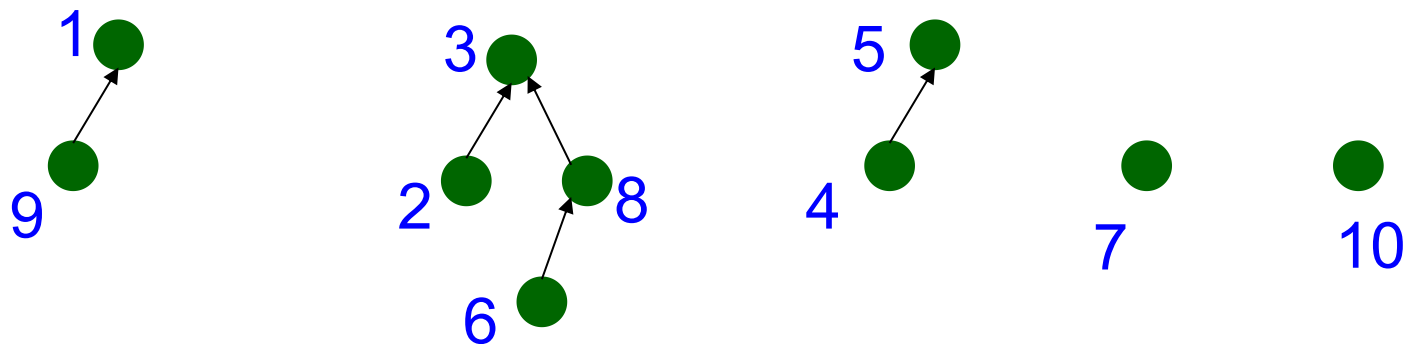
2和3所在的子集合并后

0	3	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10

3.3 不相交集数据结构

若干个子集合并后，

0	3	0	5	0	8	0	3	1	0
1	2	3	4	5	6	7	8	9	10



➤ 进行FIND、UNION操作的方法

3.3 不相交集数据结构

0	3	0	5	0	8	0	3	1	0
1	2	3	4	5	6	7	8	9	10

➤ MAKESET的时间复杂度 $O(n)$

➤ 每次 FIND 和 UNION 的时间复杂度 $O(\log n)$

7.3 最小生成树(Kruskal算法)

➤ 问题： $G=(V, E)$ 无向连通带权图，
求 (V, T) ，其中 T 是 E 的子集

回顾一下树的几条性质：(P53)

定理2.1 (a)、(b)、(c)

定理2.1. 如果 T 是有 n 个顶点的树，那么

(a) T 中任意两个顶点有惟一的一条路径连通；

(b) T 恰有 $n - 1$ 条边；

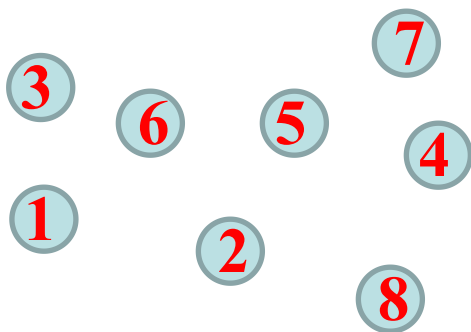
(c) 在 T 中加上一条边将构成一个回路。

7.3 最小生成树(Kruskal算法)

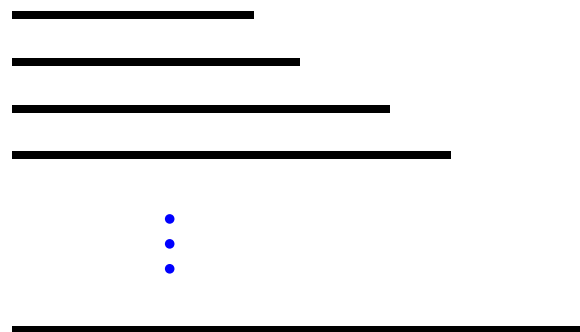
➤ 问题： $G=(V, E)$ 无向连通带权图，求 (V, T)

Kruskal算法的主要思路：

顶点集



边集



7.3 最小生成树(Kruskal算法)

- 怎么判断加入某条边，是否会导致回路？
- 思路：将节点分成若干个子集，已联通的节点放在同一子集里

MAKESET() //初始状态

0	0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10

7.3 最小生成树(Kruskal算法)

- 怎么判断加入某条边，是否会导致回路？
- 思路：将节点分成若干个子集，已联通的节点放在同一子集里

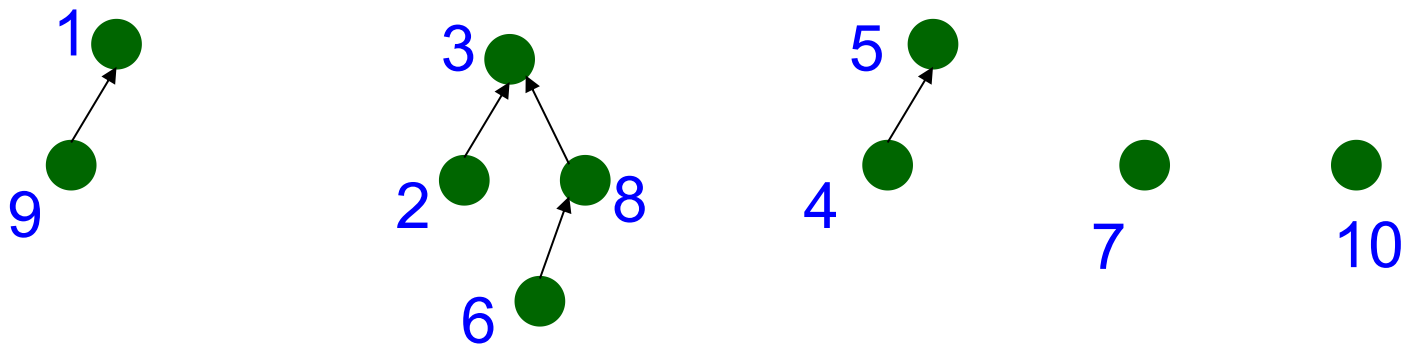
边(2, 3)加入后，

0	3	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9	10

7.3 最小生成树(Kruskal算法)

- 怎么判断加入某条边，是否会导致回路？
 - 思路：将节点分成若干个子集，已联通的节点放在同一子集里
- 加入若干个边后，

0	3	0	5	0	8	0	3	1	0
1	2	3	4	5	6	7	8	9	10

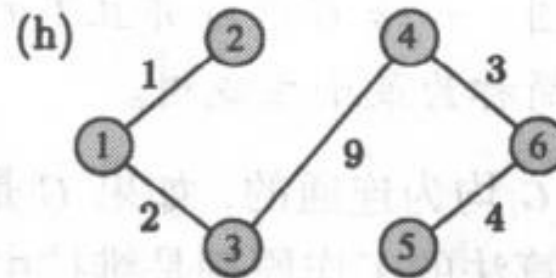
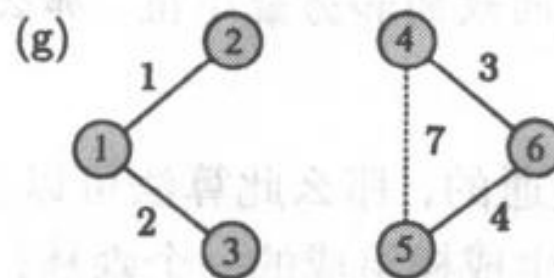
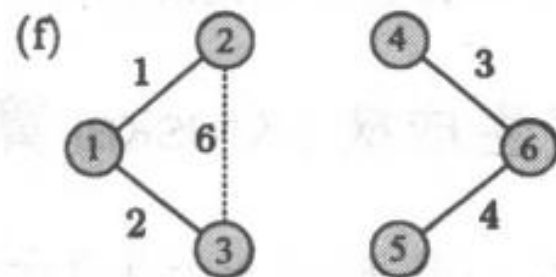
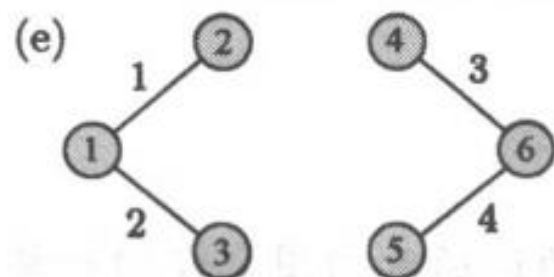
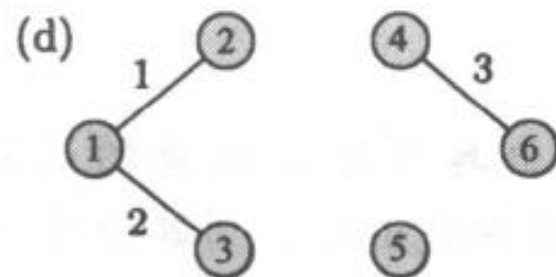
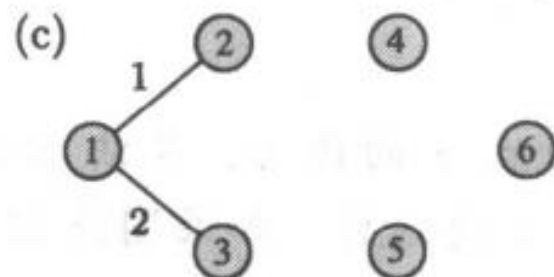
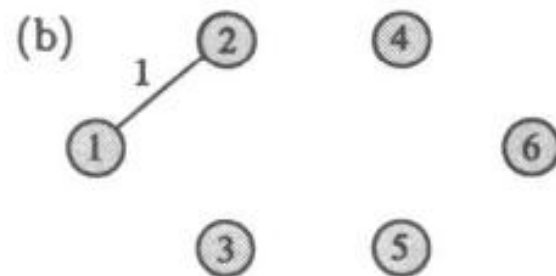
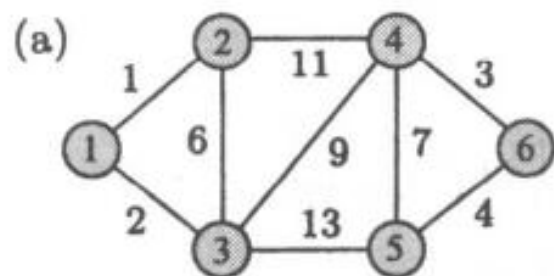


7.3 最小生成树(Kruskal算法)

➤ 问题: $G=(V, E)$ 无向连通带权图, 求
 (V, T)

Kruskal算法:

```
按非降序权重将E中的边排序;  
for(每个顶点  $v \in V$ )  
    MAKESET( $\{v\}$ );  
T= 空集;  
while ( $|T| < n - 1$ ) {  
    令  $(x, y)$  为 E 中的下一条边;  
    if ( $\text{FIND}(x) \neq \text{FIND}(y)$ ) {  
        将  $(x, y)$  加入 T;  
        UNION( $x, y$ ); } //endif  
    } //endwhile
```



7.3 最小生成树(Kruskal算法)

➤ 时间复杂性:

$O(m \log m)$

按非降序权重将E中的边排序;

for(每个顶点 $v \in V$)

 MAKESET($\{v\}$);

$O(n)$

T= 空集;

while ($|T| < n - 1$) {

 令(x,y)为 E 中的下一条边;

if (FIND(x) $\neq$ FIND(y)) {

 将(x,y)加入T;

 UNION(x,y); } //endif

}//endwhile

$O(m \log n)$

$O(m \log m)$

7.3 最小生成树(Kruskal算法)

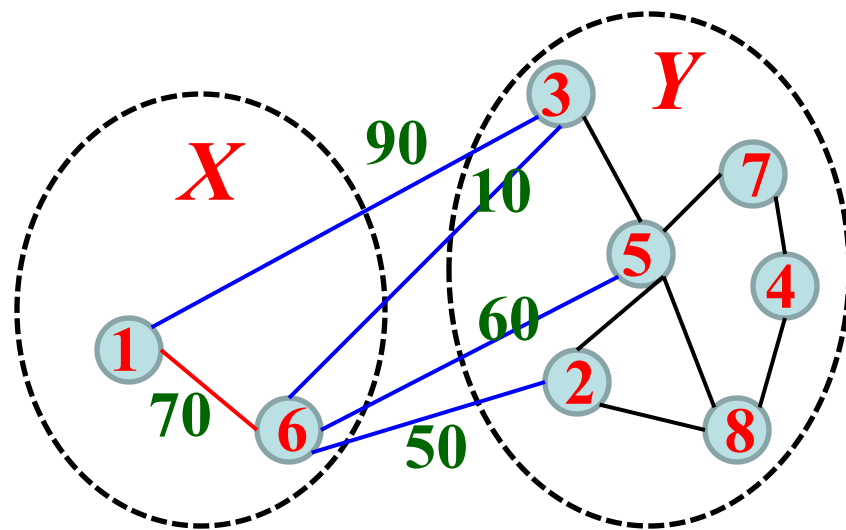
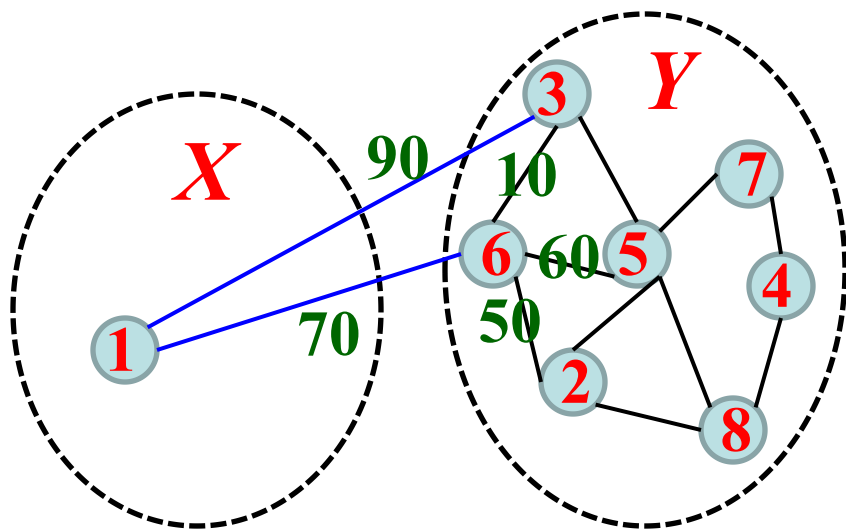
➤ 正确性：归纳法+反证法

- 1、假设Kruskal得到的结果是 T ，真正的最小生成树是 T^* ；刚开始 T 为空， T 是 T^* 的子集；
- 2、因此，我们假设在加入某个边 e 之前， T 是 T^* 的子集。假设 e 是当前不会引起回路且权值最小的边，而且边 e 的两个顶点是 x 和 y 。
- 3、如果 T^* 中不包含 e ，假设在 T^* 中， x 到 y 的路径为 $x \rightarrow v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_i \rightarrow y$ ，则该路径中至少存在一条边的权值大于 e ，记为 e' ，而且该路径和 e 合在一起会构成回路，因此，用 e 代替 e' 可以得到权值更小的生成树。矛盾！

7.4 最小生成树(Prim算法)

➤ 问题： $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法的主要思路：



7.4 最小生成树(Prim算法)

➤ 问题: $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法的框架:

$T \leftarrow \{ \}; X \leftarrow \{1\}; Y \leftarrow V - \{1\}$

while($Y \neq \text{空集}$) {

 设 (x, y) 是最小权重的边, 其中 $x \in X, y \in Y$;

$X \leftarrow X \cup \{y\}$;

$Y \leftarrow Y - \{y\}$;

$T \leftarrow T \cup \{(x, y)\}$;

}//endwhile

7.4 最小生成树(Prim算法)

➤ 问题: $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法的实现:

数组 $N[1, 2, \dots, n]$ 和 $C[1, 2, \dots, n]$, $C[y]$ 表示边 $(y, N[y])$ 的权值

7.4 最小生成树(Prim算法)

➤ 问题： $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法的步骤 (1) : 初始化

$T \leftarrow \text{空集}; X \leftarrow \{1\}; Y \leftarrow V - \{1\}$

for($y \leftarrow 2$ **to** n)

if (y 邻接于 1) **then**

$N[y] \leftarrow 1;$

$C[y] \leftarrow c[1, y];$

else $C[y] \leftarrow \infty;$

7.4 最小生成树(Prim算法)

➤ 问题: $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法的步骤 (2) :

for(j ← 1 to n-1) //寻找 n-1 条边

 令 $y \in Y$, 使得 $C[y]$ 最小;

$T \leftarrow T \cup \{(y, N[y])\}$; //将边 $(y, N[y])$ 加入 T

$X \leftarrow X \cup \{y\}$;

$Y \leftarrow Y - \{y\}$;

 for 每个邻接于 y 的顶点 $w \in Y$

 if $c[y, w] < C[w]$ then

$N[w] \leftarrow y$;

$C[w] \leftarrow c[y, w]$;

 end if

 end for

end for

7.4 最小生成树(Prim算法)

➤ 问题: $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法分析

时间复杂度:

7.4 最小生成树(Prim算法)

➤ 问题： $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法的步骤 (1) : 初始化

$T \leftarrow \text{空集}; X \leftarrow \{1\}; Y \leftarrow V - \{1\}$

for($y \leftarrow 2$ **to** n)

if (y 邻接于 1) **then**

$N[y] \leftarrow 1;$

$C[y] \leftarrow c[1, y];$

else $C[y] \leftarrow \infty;$

$O(n)$

7.4 最小生成树(Prim算法)

➤ 问题： $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法的步骤 (2) :

for($j \leftarrow 1$ to $n-1$) //寻找 $n-1$ 条边

 令 $y \in Y$, 使得 $C[y]$ 最小;

$T \leftarrow T \cup \{(y, N[y])\}$; //将边 $(y, N[y])$ 加入 T

$X \leftarrow X \cup \{y\}$;

$Y \leftarrow Y - \{y\}$;

$O(n^2)$

 for (每个邻接于 y 的顶点 $w \in Y$)

 if $c[y, w] < C[w]$ then

$N[w] \leftarrow y$;

$C[w] \leftarrow c[y, w]$;

 end if

$O(m)$

 end for

end for

7.4 最小生成树(Prim算法)

➤ 问题： $G=(V, E)$ 无向连通带权图, 求 (V, T)

Prim算法分析

时间复杂度： $O(m+n^2) = O(n^2)$